The report, which should:
– describe your overall approach to implementing the game,
– state and justify the adjustments and modifications to the initial design of the project (shown in class diagrams and use cases from Phase 1),
– explain the management process of this phase and the division of roles and responsibilities,
– list external libraries you used, for instance for the GUI and briefly justify the reason(s) for choosing
the libraries,
– describe the measures you took to enhance the quality of your code,
– and discuss the biggest challenges you faced during this phase.

Phase 2: Implementation
Game: Dungeon Escape

Joseph Trieu – 301538383
Cody Nguyen – 301612682
Niyousha Aminafshari – 301465214
Usaid Syed - 301655464
Nov 5th, 2025
CMPT276

1. **1. Overall Approach:**

In this game, the Agile development approach is used for the development of Dungeon Escape. Instead of building the entire game in a single phase which would be hard to test and debug later during the implementation time. The project is divided into smaller development cycles, each dedicated to implementing and refining a specific feature of the game such as player movement, enemy interactions, or dungeon room generation. This allows for frequent testing, feedback, and refinement. Throughout the project, new ideas can be incorporated without disrupting the existing codebase, allowing for a more dynamic and creative development process. In addition to game logic, aspects of the UI design have been created using similar method where each of the layers are created independently. The layered UI approach creates an easy workflow where testing and maintaining can be done without negative impacts on the rest of the game.

2. **Modifications:**

It was decided to separate combat from the entity package as it was becoming rather large and combat seemed deserving of its own class. The whole item package was scrapped due to time constraints. We decided to scrap the wall and room classes, as it would redundant down the line. Since we rather just use the tile (renamed from cell) to store whether there is a collision on the specific tile or not. As such keeping the individual classes for door, wall, etc. Would simply just lead to boiler plate code and may lead to issues with scalability. As our current design emphasizes a more clear and concise way to handle how the map is loaded and used throughout the code base. Similarly for the UI design we created initially, we decided to create a simpler model for phase 2 due to both time constraints and learning the design aspects of the language. Currently, the design follows the title of the game at the top with some simple buttons for zooming in, pausing and replaying the game etc., as well as the dialogue bar for commentary purposes with a work in progress menu button for extra features. The features we modified for this phase of the project can be added in for the next phase as well.

3. **Management Process:**

We split the different sections of the project to each group member, to work on independently. Throughout the week or the at the halfway deadlines we would set up meetings to discuss our progress into the game and what needs to be done for second phase. Usaid handled the framework, asset accruement, environments, and was our level designer. Niyousha handled all UI related tasks and is our UI designer. Joseph handled code related to loading in our maps and collision. Cody handled creating Enemy and Combat classes including pathfinding. For managing the project on the repo, we made sure to communicate before pushing and modifying files that would cause collisions or chatted about if we were running into problems implementing our part of the assignment.

4. **External Library:**

We chose to use java.swing for the Gui as it was a simple ready to use gui library in java. Aside from Swing, only standard internal libraries were used in the development of this game.

5. **Evaluation:**

Some measures we took to ensure code quality was creating branches as to not disrupt main. We also adhered to Object oriented Principles and implemented interface to help keep our code organized and modular. We also emphasized scalability by introducing (factory methods) to make updating existing parts of our project easier and more efficient.

In this phase, the core features of Dungeon Escape have been successfully implemented and tested. The player can navigate through interconnected dungeon rooms represented on a 2D grid. Each room is procedurally generated and contains one main player, enemies, and interactive elements such as doors, and walls. For the UI aspect of the game, the main code has not been adjusted to fully add in the UI layer as there are more components to be added. But each sepearet button or feature has been tested to run. The user can press buttons to pause a game, or zoom in and out of the screen.

Description:

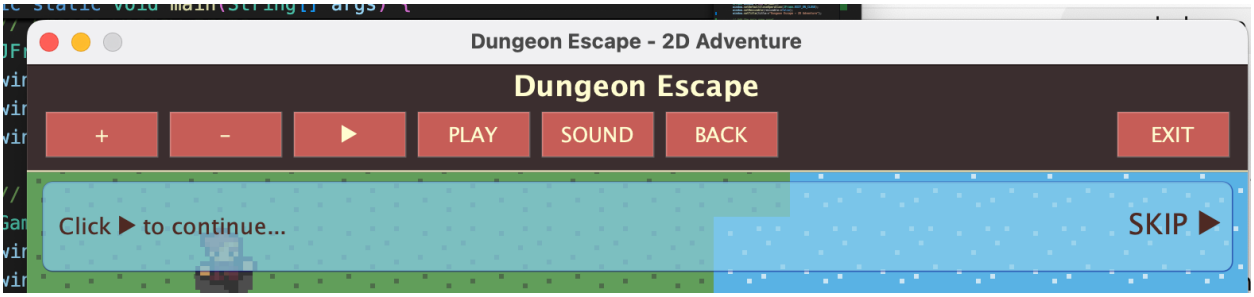
Player movement: The main character is able to move freely between different rooms and interact with elements on the map. Collision detection prevents movement through walls or barriers.

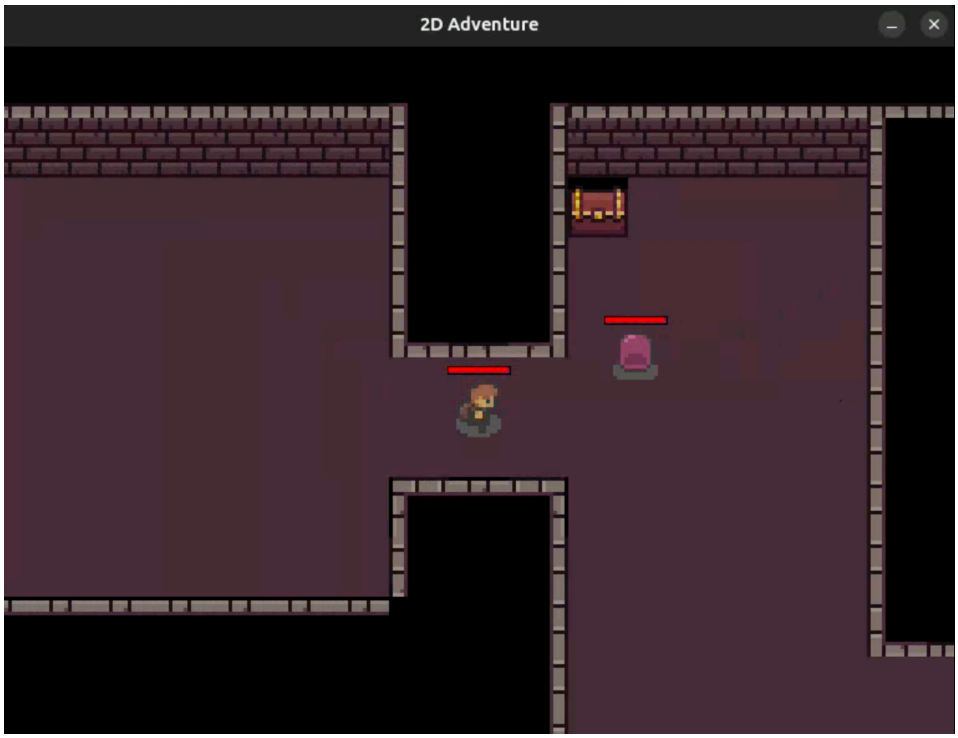
Enemies: Basic enemy entities follow simple pathfinding logic to detect and chase the player. If the player collides with an enemy, the main player lose health.

Attacking system: The player can engage in combat with certain enemies.

Map loading: Rooms and tiles load dynamically as the player moves through doors, creating a continuous dungeon-like experience.

UI: Created the basic layout of the top panel featuring zoom in and out, play and sound button including the title of the game at the very top. Underneath the panel there's a dialogue bar where we would include what the game prompts in the main file and testing stages of the game. The dialogue bar also comes with a feature where the prompts can be skipped if the user chooses to.





6. **Challenges**:

A major challenged some of our group mates faced was using git as this was their first time.

Besides, the biggest struggle is getting used to the design aspect of java and becoming familiar with Java Swing to create the design. The trouble shooting of each design is also a process that takes some time as each change or addition to the screen should be checked after created. As a result, there was not enough time to get some of the extra features we were hoping to create by the end of phase 2 such as having a functioning menu button with sound and settings to click.